

Desafio PUC 2026 - Dia 2: Repetição

05/10/2026 | for, range(), while e percorrer listas | 2 fáceis, 2 médias, 1 estilo OBI | tempo sugerido: 1h30 a 2h

Como usar

1. Resolva **sem consulta**. Se travar, anote onde travou e me diga; a ajuda vem em degraus.
2. Antes de escrever código, responda mentalmente as 3 perguntas de cada questão.
3. **Rode os 4 exemplos** de cada questão e compare a saída **letra por letra** com a tabela.
4. Invente **dois casos seus** (um deles nos limites) e confira na mão o que o programa deve imprimir.
5. Envie o código de cada questão conforme terminar. Pode mandar uma de cada vez.

Hábitos do seu registro para vigiar hoje: copiar as palavras de saída do enunciado (POSITIVO, NEGATIVO, ZERO, FOGO!, NENHUM, SIM, NAO sem acento); **int(input())** sem texto dentro do parênteses; rodar os exemplos antes de enviar; decidir a ordem das perguntas (if/elif) **antes** de escrever.

Ferramenta de leitura (vale para as questões com lista). Quando a segunda linha da entrada tem vários números separados por espaço, use: `lista = list(map(int, input().split()))`
`input()` lê a linha como texto; `.split()` separa nos espaços; `map(int, ...)` converte cada pedaço para inteiro; `list(...)` junta tudo numa lista. Teste com `print(lista)` antes de continuar e apague depois.

Duas perguntas-chave do dia:

(a) O que acontece **A CADA repetição** e o que acontece **DEPOIS do loop**? (b) No while: **o que muda a cada repetição e faz o loop terminar**?

Questão 1 (fácil): Contagem regressiva

Um foguete vai decolar e o controle de missão faz a contagem regressiva. Dado um inteiro **N**, imprima os números de N até 1, em ordem decrescente, um por linha. Depois que a contagem terminar, imprima uma única vez a palavra **FOGO!**

Entrada: uma linha com um inteiro N ($1 \leq N \leq 100$).

Saída: os números de N até 1, um por linha, e em seguida a linha FOGO!

Exemplos

#	Entrada	Saída
1	1	1 FOGO!
2	3	3 2 1 FOGO!
3	5	5 4 3 2 1 FOGO!
4	2	2 1 FOGO!

Antes de programar, pense:

1. Qual é a entrada e qual é a saída? Quantas linhas saem quando $N = 1$?
2. O que acontece a cada repetição? O que acontece uma única vez, depois do loop?
3. Como o range() faz para contar de trás para frente? O 1 entra? O 0 entra? Teste na mão com $N = 3$.

Questão 2 (fácil): Sinal de cada número

Uma máquina registrou uma sequência de números. Para cada número, na mesma ordem em que aparecem, imprima uma linha dizendo se ele é **POSITIVO**, **NEGATIVO** ou **ZERO**.

Entrada: a primeira linha tem um inteiro N ($1 \leq N \leq 100$). A segunda linha tem N inteiros separados por espaço, cada um entre -1000 e 1000.

Saída: N linhas, cada uma com **POSITIVO**, **NEGATIVO** ou **ZERO**.

Exemplos

#	Entrada	Saída
1	4 5 -3 0 8	POSITIVO NEGATIVO ZERO POSITIVO
2	1 0	ZERO
3	3 -1 -2 -3	NEGATIVO NEGATIVO NEGATIVO
4	5 10 20 0 0 -7	POSITIVO POSITIVO ZERO ZERO NEGATIVO

Treino pedido: percorra a lista com `for elemento in lista:`, sem índice manual.

Antes de programar, pense:

1. O que o programa faz com cada elemento? E precisa fazer algo depois do loop?
2. Quantos casos existem? Um mesmo número pode ser dois casos ao mesmo tempo? Em que ordem você pergunta?
3. Qual é o limite mais perigoso: o zero? Em qual exemplo ele aparece?

Questão 3 (média): Quantos dígitos?

Dado um inteiro positivo **N**, descubra quantos dígitos ele tem. **Regra desta questão:** não use `str()` nem `len()`. Use um `while` e a divisão inteira (`//`).

Entrada: uma linha com um inteiro **N** ($1 \leq N \leq 1000000000$).

Saída: uma linha com a quantidade de dígitos de **N**.

Exemplos

#	Entrada	Saída
1	7	1
2	100	3
3	12345	5
4	1000000000	10

Antes de programar, pense:

1. Se eu tirar o último dígito de um número, que conta eu faço em Python? Teste na mão com 12345.
2. O que muda a cada repetição e faz o loop terminar? Em que valor ele deve parar?
3. O que acontece com $N = 7$ (um dígito) e com $N = 1000000000$ (limite)? O que precisa ser contado e quando?

Questão 4 (média): Primeiro número negativo

Uma lista de números representa o saldo de uma conta ao fim de cada dia. Descubra em **qual dia** (posição, começando em 1) aparece o **primeiro** saldo negativo. Se nenhum saldo for negativo, imprima **NENHUM**.

Entrada: a primeira linha tem um inteiro N ($1 \leq N \leq 100$). A segunda linha tem N inteiros separados por espaço, entre -1000 e 1000.

Saída: uma linha com a posição do primeiro negativo, ou NENHUM.

Exemplos

#	Entrada	Saída
1	4 3 5 -2 7	3
2	3 4 0 9	NENHUM
3	2 -1 -2	1
4	1 -5	1

Antes de programar, pense:

1. A posição que a saída pede começa em 1 ou em 0? E o índice da lista em Python?
2. Preciso olhar a lista inteira ou posso parar quando achar o primeiro? E se existirem vários negativos, qual vale?
3. O que imprimir quando o loop termina e não achou nenhum? Esse print vem dentro ou depois do loop?

Questão 5 (estilo OBI): Fila bem organizada

Na escola do Caio, a professora organiza os alunos em uma fila e quer saber se a fila está **estritamente crescente** em altura, isto é, cada aluno é mais alto que o aluno imediatamente à sua frente na fila (o anterior). Alturas iguais **quebram** a regra. Dada a lista de alturas, na ordem da fila, diga se a fila está bem organizada.

Entrada: a primeira linha tem um inteiro N ($1 \leq N \leq 100$), o número de alunos. A segunda linha tem N inteiros separados por espaço, as alturas em cm (entre 100 e 220).

Saída: uma linha com SIM se a fila está estritamente crescente, ou NAO (sem acento) caso contrário.

Exemplos

#	Entrada	Saída
1	3 1 2 3	SIM
2	3 1 3 3	NAO
3	1 5	SIM
4	2 9 4	NAO

Antes de programar, pense:

1. O que eu preciso comparar? Cada elemento sozinho basta ou preciso de dois elementos ao mesmo tempo?
2. Basta uma única falha para a resposta ser NAO? Posso decidir antes de acabar o loop? Quando imprimo SIM: a cada repetição ou depois de analisar tudo?
3. Quantas comparações existem com N alunos? O que acontece com $N = 1$, em que não há ninguém para comparar? Que valor de `range()` é seguro?

Checklist antes de enviar cada questão: (1) rodei os 4 exemplos; (2) comparei cada print letra por letra com a tabela; (3) testei um limite ($N = 1$ ou valor máximo); (4) o print está no lugar certo (dentro ou depois do loop); (5) o while, se houver, termina.